

PocketSmart AI: Your Smart Budget

Main Functionality Code Routings

NAAN MUDALVAN PROJECT

20-Page Detailed Project PDF

A complete technical document covering the main functionality flow, code routing, system modules, testing, documentation, and demonstration.

1. Project Overview and Main Functionality Code Routings

PocketSmart AI: Your Smart Budget is the main project title for the Naan Mudhalvan project. The project is designed as a smart budgeting application that accepts income, expenses, savings goals, and domain-specific budget information and converts those inputs into useful financial summaries. The subtitle, "Main Functionality Code Routings," represents the technical focus of this document: explaining how the major functions move through the application from user input to validation, calculation, processing, and output. The project combines a conventional budgeting engine with an intelligent explanation layer. The most important design principle is that financial calculations are performed by deterministic application logic, while AI can help explain the verified results in simple language.

- Main title: PocketSmart AI: Your Smart Budget.
- Project context: Naan Mudhalvan project.
- Subtitle: Main Functionality Code Routings.
- Primary goal: simplify budget planning.
- Input: income and financial resources.
- Input: fixed and variable expenses.
- Input: savings goals.
- Input: budget category and domain.
- Processing: validation.
- Processing: calculation.
- Processing: category aggregation.
- Processing: savings analysis.
- Processing: scenario comparison.
- Output: budget summary.
- Output: remaining balance.
- Output: category analysis.
- Output: savings status.
- Output: understandable recommendations.
- Output: reports or dashboard information.
- The routing design separates user interface, business logic, data handling, and AI explanation.
- This structure supports testing and maintenance.
- It also makes the project easier to demonstrate.
- The system can be expanded with additional domains.
- The system can support personal, student, family, travel, event, and small-business budgets.
- All major functions should have clear input, processing, and output behavior.
- The code-routing approach provides a map of how each feature works.
- The project is suitable for academic demonstration and future development.
- The following pages explain the complete functionality routing in detail.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

2. Project Objectives and Scope

The objective of PocketSmart AI is to provide a practical budgeting assistant that is easy to use and technically understandable. The project scope includes the complete flow from entering budget information to displaying calculated results. The project also demonstrates how a modular application can be organized so that each function has a clear responsibility. The scope is intentionally structured around core budgeting rather than attempting to become a complete financial institution or investment platform.

- Objective 1: create a simple budget entry interface.
- Objective 2: accept income values.
- Objective 3: accept expense values.
- Objective 4: classify expenses.
- Objective 5: calculate total income.
- Objective 6: calculate total expenses.
- Objective 7: calculate remaining balance.
- Objective 8: compare planned savings with a target.
- Objective 9: support domain-specific categories.
- Objective 10: provide understandable summaries.
- Objective 11: provide what-if budget comparison.
- Objective 12: validate incorrect inputs.
- Objective 13: keep calculations deterministic.
- Objective 14: separate AI explanation from arithmetic.
- Objective 15: support testing and documentation.
- Objective 16: make the application easy to demonstrate.
- Scope includes frontend input handling.
- Scope includes backend or service-layer routing where applicable.
- Scope includes budget calculation logic.
- Scope includes result presentation.
- Scope includes sample data for multiple domains.
- Scope does not require guaranteed financial forecasting.
- Scope does not treat AI output as professional financial advice.
- Scope can later be expanded through additional modules.
- The final scope is sufficient to demonstrate a complete software project lifecycle.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

3. System Architecture and Code Routing

The application can be implemented using a layered architecture. The user interface collects data and sends it to a controller or route handler. The route validates the request and passes it to the budget service. The budget service calls the calculation engine. The calculation engine returns trusted numerical results. The service can then call a category analyzer and goal analyzer. The resulting object is returned to the controller and displayed by the user interface. If AI explanations are enabled, the verified result object is passed to an explanation service after calculations are complete.

- UI → Budget Route → Validation → Budget Service.
- Budget Service → Calculation Engine.
- Calculation Engine → Category Analyzer.
- Calculation Engine → Goal Analyzer.
- Budget Service → Scenario Engine when required.
- Verified result → Explanation Service.
- Verified result → Report Generator.
- Service result → Controller/Route.
- Controller result → UI Dashboard.
- Optional result → Database.
- Authentication layer can protect user-specific data.
- Error handler captures invalid requests.
- Logging layer records technical events without exposing unnecessary sensitive data.
- The architecture avoids placing business calculations directly inside UI components.
- Reusable services can be tested independently.
- Routes should remain thin and delegate calculations to services.
- Models should represent structured budget data.
- Validation schemas should reject malformed values.
- The calculation engine should not depend on screen-specific code.
- The explanation service should not independently invent financial totals.
- This routing makes the project modular.
- It also supports future API development.
- It provides a clear technical story for the Naan Mudhalvan demonstration.
- The architecture can be implemented as a web, mobile, or desktop application.
- The design supports controlled future enhancement.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

4. User Registration / Start Budget Routing

The first functional route can be a start-budget or user-session route. If the project includes authentication, the user signs in before accessing saved budgets. If authentication is outside the project scope, the start screen can directly open a new budget. The purpose of this routing is to establish a clean entry point and initialize an empty or saved budget object.

- Route example: /.
- Route example: /login.
- Route example: /dashboard.
- Route example: /budget/new.
- User opens the application.
- System loads the main interface.
- User selects New Budget.
- Application creates an empty budget model.
- Default currency can be selected.
- Budget period can be selected.
- Domain can be selected.
- User can choose monthly, weekly, trip, event, or custom planning.
- Application assigns a unique budget session identifier when persistence is enabled.
- Initial fields are loaded.
- Validation rules are loaded.
- Domain categories are loaded.
- User is routed to the income-entry screen.
- Errors at this stage should be clear and non-technical.
- If login exists, unauthorized users should not access protected budgets.
- If login is not used, the application can operate as a local prototype.
- The route should not perform financial calculations.
- The route should only initialize the budgeting workflow.
- This separation keeps the routing easy to understand.
- It also makes automated UI testing simpler.
- The start route establishes the foundation for all later functionality.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

5. Income Entry Code Routing

Income entry is one of the core functionalities. The user can enter one or more income sources, such as salary, allowance, freelance income, business revenue, or other planned income. The route receives structured input, validates the amount and period, stores the entry, and sends the updated budget to the calculation service. Multiple income entries should be supported because real budgets may have more than one source.

- Route example: POST /api/budget/income.
- Request contains budgetId.
- Request contains income source.
- Request contains amount.
- Request contains frequency or period.
- Request contains optional description.
- Validation checks required fields.
- Validation checks numeric format.
- Validation checks positive or permitted zero values.
- System normalizes the amount.
- Income record is added to the budget.
- Budget total is recalculated.
- Updated result is returned.
- UI refreshes the income summary.
- User can add another income source.
- User can edit an existing income source.
- User can delete an income source.
- Duplicate entries should be allowed when they represent separate sources.
- Currency formatting should be consistent.
- Large valid values should be handled safely.
- Invalid input should return a structured error.
- The calculation engine remains the source of truth.
- The UI should never manually calculate a different total.
- Testing should cover normal, zero, invalid, and large values.
- Income routing concludes when the updated budget is available to the next step.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

6. Expense Entry Code Routing

Expense routing handles fixed and variable expenses. A fixed expense is generally planned and recurring, while a variable expense can change from period to period. The application should not force every real-world expense into only one type if the project later needs more flexibility. Each expense contains a category and amount, and the calculation engine aggregates the values.

- Route example: POST /api/budget/expenses.
- Request contains budgetId.
- Request contains category.
- Request contains expense type.
- Request contains amount.
- Request contains period.
- Validation checks category.
- Validation checks amount.
- Validation checks permitted expense type.
- Expense is stored in the current budget.
- Category total is updated.
- Overall expense total is updated.
- Remaining balance is recalculated.
- Dashboard receives updated results.
- User can edit an expense.
- User can remove an expense.
- Category list depends on selected domain.
- Personal categories can include rent, food, transport, and utilities.
- Student categories can include hostel, food, transport, and study materials.
- Travel categories can include transport, accommodation, food, and activities.
- Business categories can include rent, salaries, supplies, and marketing.
- Event categories can include venue, food, decoration, and transport.
- Testing should verify category totals reconcile with total expenses.
- Expense routing provides the main input for budget analysis.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

7. Budget Calculation Engine Routing

The calculation engine is the most important computational component. It should be deterministic and independently testable. The engine receives validated data and calculates totals using explicit formulas. It should not depend on AI. This allows the project to demonstrate that the core budget mathematics remains reliable even if the explanation layer is changed or disabled.

- Input: validated income list.
- Input: validated expense list.
- Input: savings target.
- Input: optional scenario values.
- Calculate total income.
- Calculate fixed expenses.
- Calculate variable expenses.
- Calculate total expenses.
- Calculate remaining balance.
- Calculate planned savings when supported.
- Calculate savings gap.
- Calculate expense ratio when income is greater than zero.
- Calculate category shares when total expenses are greater than zero.
- Generate a structured result object.
- Example formula: $\text{totalIncome} = \text{sum}(\text{income.amount})$.
- Example formula: $\text{totalExpenses} = \text{sum}(\text{expense.amount})$.
- Example formula: $\text{balance} = \text{totalIncome} - \text{totalExpenses}$.
- Example formula: $\text{savingsGap} = \text{targetSavings} - \text{plannedSavings}$.
- Guard against division by zero.
- Apply consistent rounding only at the display boundary where possible.
- Return calculations with clear field names.
- Do not allow AI-generated values to replace calculation results.
- Unit tests should directly target this engine.
- Every dashboard total should originate from this engine.
- The calculation route is the trusted center of PocketSmart AI.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

8. Category Analysis Code Routing

Category analysis converts a list of expenses into meaningful groups. This function allows the dashboard to show where the budget is being used. Category analysis should be deterministic because the values are derived from entered expenses. The AI layer may later describe the category distribution, but it should receive the computed category totals rather than guessing them.

- Input: validated expense list.
- Group records by category.
- Sum amounts for each category.
- Sort categories by amount where required.
- Calculate category share.
- Identify the largest categories.
- Identify categories with unusually high values only when rules are defined.
- Return category summary.
- Return total expenses.
- Return category count.
- UI can display a table.
- UI can display a bar chart.
- UI can display a pie or doughnut chart.
- Student template can emphasize education-related categories.
- Family template can emphasize household categories.
- Travel template can emphasize trip categories.
- Business template can emphasize operating costs.
- Event template can emphasize event-service categories.
- Category names should remain consistent.
- Unknown categories can be accepted if custom categories are supported.
- Tests should confirm that category totals equal overall expenses.
- Tests should confirm that deleting an expense updates the category result.
- Tests should confirm that editing an expense moves the correct amount.
- Category routing makes raw expenses easier to understand.
- It is a major input to the explanation and reporting layers.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

9. Savings Goal Code Routing

Savings goals allow the user to compare planned savings with a desired target. The goal route accepts a target amount and connects it with the budget's available balance or planned savings field. The project should clearly define whether “planned savings” means an explicit savings expense or the remaining balance after expenses. The selected definition must be consistent across the interface, calculations, tests, and documentation.

- Route example: POST /api/budget/goals.
- Input: target amount.
- Input: target period.
- Input: optional goal name.
- Validate target amount.
- Store goal information.
- Calculate available balance.
- Calculate planned savings if explicitly entered.
- Calculate savings gap.
- Return goal status.
- Status can be defined as on-track, below-target, or above-target according to numeric rules.
- Do not use subjective AI judgments for the basic status.
- Display target.
- Display planned amount.
- Display difference.
- Display progress percentage when denominator is valid.
- Allow goal editing.
- Allow goal deletion.
- Support more than one goal only if project scope permits.
- Test target = zero.
- Test target greater than available amount.
- Test planned savings equal to target.
- Test planned savings above target.
- Test missing target.
- Savings routing connects budgeting with goal planning.
- It gives the dashboard another useful output beyond simple totals.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

10. What-If Scenario Code Routing

The what-if feature lets the user change one or more budget values and compare the result with a baseline. This is useful for demonstrations because it visibly shows how the budget reacts to changes. The scenario engine should copy the baseline rather than modifying the original budget until the user chooses to save the scenario.

- Route example: POST /api/budget/scenario.
- Input: baseline budget.
- Input: changed income or expense values.
- Create a temporary scenario object.
- Run the calculation engine on the scenario.
- Calculate baseline balance.
- Calculate scenario balance.
- Calculate difference.
- Calculate expense difference.
- Calculate savings difference.
- Return baseline and scenario results.
- Display comparison table.
- Highlight changed categories.
- Allow reset to baseline.
- Allow scenario save only if supported.
- Scenario calculations must use the same calculation engine.
- Do not duplicate formulas in the scenario module.
- Test an increased expense.
- Test a reduced expense.
- Test increased income.
- Test multiple changed values.
- Test unchanged scenario.
- Test invalid scenario input.
- What-if routing provides a strong demonstration of intelligent budgeting behavior.
- It also helps users understand the impact of their assumptions.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

11. AI Explanation Code Routing

The AI explanation layer is responsible for converting verified numerical results into readable observations. The AI should not be the authority for arithmetic. The application first completes deterministic calculations and then sends a structured result to the explanation component. The explanation should clearly distinguish calculated facts from optional suggestions. This approach reduces the risk of the AI generating a number that does not match the application.

- Input to AI: total income.
- Input to AI: total expenses.
- Input to AI: remaining balance.
- Input to AI: category totals.
- Input to AI: savings target.
- Input to AI: scenario difference.
- AI generates a summary.
- AI explains major categories.
- AI can explain the effect of a scenario.
- AI can suggest budgeting questions.
- AI should not invent transactions.
- AI should not alter calculation results.
- AI should not claim certainty about future financial outcomes.
- AI should not expose private information unnecessarily.
- AI output should be reviewed for contradictions.
- Application can compare important displayed numbers with the trusted result object.
- Fallback message should be available if AI service fails.
- The application remains usable without AI.
- AI latency should not block basic calculations when avoidable.
- Prompt templates should be version controlled where applicable.
- Test examples should include correct, contradictory, incomplete, and service-failure situations.
- The AI route is therefore an enhancement layer.
- The trusted financial logic remains in the application.
- This separation is a major technical feature of PocketSmart AI.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

12. Dashboard and Result Routing

The dashboard is where the processed information becomes understandable to the user. It should present the most important numbers first and allow the user to explore details. The dashboard should be generated from the trusted result object so that all visible values remain consistent. The interface can include summary cards, category charts, savings progress, scenario comparison, and an AI-generated explanation section.

- Route example: GET /api/budget/:id/summary.
- Load budget data.
- Validate access.
- Run or retrieve calculations.
- Load category analysis.
- Load goal analysis.
- Load scenario result when applicable.
- Load AI explanation when available.
- Build dashboard response.
- Display total income.
- Display total expenses.
- Display remaining balance.
- Display savings target.
- Display category breakdown.
- Display largest expense categories.
- Display what-if comparison.
- Display AI explanation separately from calculated values.
- Provide edit buttons.
- Provide reset or start-new-budget action.
- Provide export/report option when available.
- Use readable currency formatting.
- Use clear positive, neutral, and warning states.
- Do not hide important calculation assumptions.
- Dashboard routing is the final presentation stage of the core budget workflow.
- It is also the main screen used during the Naan Mudhalvan demonstration.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

13. Domain Template Routing

PocketSmart AI becomes more useful when the same core engine can support different budget domains. A domain template contains category suggestions, optional fields, labels, and demonstration data. The template should not duplicate the calculation engine. Instead, it supplies configuration that the common engine consumes.

- Route example: GET /api/domains.
- Return supported domains.
- User selects a domain.
- Route example: GET /api/domains/:domain/categories.
- Return suggested categories.
- Personal: rent, food, transport, utilities, education.
- Student: hostel, food, transport, books, study materials.
- Family: rent, groceries, utilities, transport, education, healthcare.
- Travel: transport, accommodation, food, activities, emergency reserve.
- Business: revenue, rent, salaries, supplies, utilities, marketing.
- Event: venue, food, decoration, transport, equipment, contingency.
- Custom categories can be added if supported.
- Templates should be version controlled.
- Templates should be tested for missing categories.
- Domain changes should not erase existing data without confirmation.
- The calculation engine remains common to all domains.
- The UI can adjust labels based on the template.
- AI explanations can mention the selected domain.
- Test every domain with at least one realistic dataset.
- Domain routing demonstrates reuse.
- It reduces duplicated code.
- It also makes future project expansion easier.
- New domains can be added as configuration plus tests.
- This is an important scalability feature.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

14. Validation and Error Handling Routing

Validation protects the calculation engine from malformed data. Validation should happen before financial calculations. Errors should be structured and understandable. A user should know what field needs correction and why. Technical details such as stack traces should not be shown to ordinary users. The same validation rules should be used consistently across UI and server-side boundaries when a backend exists.

- Validate required fields.
- Validate numeric fields.
- Validate currency and period.
- Validate expense category.
- Validate allowed expense type.
- Reject malformed requests.
- Handle missing budget identifiers.
- Handle unauthorized requests.
- Handle nonexistent budgets.
- Handle duplicate or conflicting data where applicable.
- Prevent invalid division.
- Prevent unsupported negative amounts when the business rule prohibits them.
- Allow zero where the business rule permits it.
- Handle very large valid numbers safely.
- Return clear error codes or messages.
- Log technical errors securely.
- Do not expose internal stack traces.
- Do not reveal unnecessary sensitive information.
- Test validation independently.
- Test validation through the full route.
- Test error responses in the UI.
- Provide recovery guidance such as “Enter a valid amount.”
- Error handling should keep the application stable.
- Validation routing is essential for both correctness and user experience.
- It is also a major area for testing during evaluation.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

15. Database and Data Routing

If the project includes persistent storage, the database stores structured budget information. A relational database can contain users, budgets, income records, expense records, goals, and scenarios. A document database can represent a budget as a nested object. The choice depends on the implementation stack. Regardless of database type, the routing should separate data access from business calculations.

- Budget record contains identifier.
- Budget record contains domain.
- Budget record contains period.
- Income records contain source and amount.
- Expense records contain category and amount.
- Goal records contain target and period.
- Scenario records contain baseline reference and changes when saved.
- Repository or data-access layer handles database operations.
- Service layer handles business rules.
- Calculation engine handles mathematics.
- Routes call services rather than writing raw database logic everywhere.
- Transactions can protect multi-step updates when supported.
- Indexes can improve lookup by budget identifier.
- Access control must be applied to user-owned data.
- Input must be validated before persistence.
- Sensitive data should be minimized.
- Backups should be considered for production deployment.
- Deletion behavior should be documented.
- Data migration should be considered for future versions.
- Database errors should be converted into safe application errors.
- Tests can use a test database or mock repository.
- Seed data can support demonstrations.
- Database routing makes the project persistent and scalable.
- The exact implementation can vary without changing the business rules.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

16. Testing and Main Functionality Verification

Testing verifies the main code routings described in this document. Each route should have unit, integration, and end-to-end evidence where appropriate. The most important numerical functions should have deterministic tests. Route tests should verify request validation, service invocation, result structure, and error behavior. UI tests should verify that users can complete the budget workflow.

- Test income route with valid amount.
- Test income route with missing amount.
- Test expense route with valid category.
- Test expense route with invalid amount.
- Test calculation engine with known totals.
- Test calculation engine with zero income.
- Test calculation engine with empty expense list.
- Test category analysis reconciliation.
- Test savings goal comparison.
- Test what-if increased expense.
- Test what-if increased income.
- Test domain template loading.
- Test dashboard result consistency.
- Test AI service success.
- Test AI service failure and fallback.
- Test unauthorized budget access.
- Test nonexistent budget.
- Test edit expense behavior.
- Test delete expense behavior.
- Test reset behavior.
- Test report/export values against trusted calculations.
- Test large valid values.
- Test repeated requests where idempotency matters.
- Record expected and actual results.
- Record defects and fixes.
- Final testing conclusion: the application should be demonstrated only after core routes have passed the defined verification set.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

17. Documentation and Demonstration Routing

The project documentation should explain not only what the application does but also how the code routes connect. The demonstration can use the same routing map as the technical explanation. This makes the presentation easier for evaluators to follow: user action leads to route, route leads to service, service leads to calculation, and result leads to the dashboard.

- Document project title and subtitle.
- Document project objective.
- Document target users.
- Document scope.
- Document architecture.
- Document route list.
- Document request and response structures.
- Document calculation formulas.
- Document domain templates.
- Document database design if used.
- Document validation rules.
- Document error-handling behavior.
- Document AI explanation boundaries.
- Document test cases.
- Document sample inputs.
- Document expected outputs.
- Demonstration begins at the start screen.
- Demonstration selects a domain.
- Demonstration enters income.
- Demonstration enters expenses.
- Demonstration shows calculated results.
- Demonstration changes one value.
- Demonstration shows scenario comparison.
- Demonstration displays AI explanation.
- Demonstration shows another domain.
- Demonstration concludes with testing, documentation, limitations, and future scope.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

18. Sample Main Functionality Code Routing

The following simplified pseudo-routing illustrates the main flow. It is intentionally technology-neutral so that the same concept can be implemented using Python, Java, JavaScript, Node.js, PHP, .NET, or another suitable stack. The important concept is separation of routing, validation, services, calculation, and presentation.

- POST /api/budget → createBudget().
- POST /api/budget/income → addIncome().
- POST /api/budget/expenses → addExpense().
- PUT /api/budget/expenses/:id → updateExpense().
- DELETE /api/budget/expenses/:id → deleteExpense().
- POST /api/budget/goals → setGoal().
- POST /api/budget/scenario → runScenario().
- GET /api/budget/:id/summary → getSummary().
- GET /api/domains → listDomains().
- GET /api/domains/:id/categories → getCategories().
- POST /api/budget/:id/explanation → explainBudget().
- POST /api/budget/:id/report → generateReport().
- Route → validateRequest().
- Route → budgetService.
- budgetService → calculationEngine.
- calculationEngine → resultObject.
- resultObject → categoryAnalyzer.
- resultObject → goalAnalyzer.
- resultObject → scenarioAnalyzer when required.
- resultObject → explanationService when requested.
- Final response → dashboard.
- Every route should have success and error behavior.
- Every core route should have automated tests.
- This routing list can be converted into actual controller code.
- It provides the central technical map for the project subtitle: Main Functionality Code Routings.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

19. Final Project Evaluation and Conclusion

PocketSmart AI: Your Smart Budget is structured as a complete application concept with a clear technical routing model. The project demonstrates how a user action can move through validation, business services, deterministic calculations, analysis, AI explanation, and dashboard presentation. The eight major project areas—brainstorming and ideation, requirement analysis, project design, project planning, development, testing, documentation, and demonstration—are connected through the same architecture. The result is a project that can be explained both from a user perspective and from a software-engineering perspective.

- The project begins with a real-world budgeting problem.
- The idea is transformed into measurable requirements.
- The design separates major responsibilities.
- The plan organizes development work.
- The code-routing model explains how functions connect.
- The calculation engine provides trusted numerical results.
- The category engine explains spending structure.
- The savings engine supports goal comparison.
- The scenario engine supports what-if analysis.
- The domain engine supports different real-world budgets.
- The AI layer provides readable explanations.
- The dashboard communicates the results.
- Validation protects the system from invalid input.
- Testing provides evidence for the main functions.
- Documentation preserves the technical design.
- Demonstration connects all components into one story.
- The project can be extended with additional features.
- The project can be adapted to different technology stacks.
- The architecture supports maintainability.
- The architecture supports future testing.
- The project can be submitted as a structured Naan Mudhalvan software project.
- The project title and subtitle clearly identify its purpose.
- The final evaluation should be based on implemented functionality, evidence, testing, and presentation quality.
- The overall project demonstrates practical software development skills.
- The conclusion is that PocketSmart AI provides a strong foundation for a smart-budget application and a clear route-by-route technical explanation.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.

20. Final Summary, Deliverables, and Presentation Checklist

This final page summarizes the complete project package. The main title is “PocketSmart AI: Your Smart Budget” and the technical subtitle is “Main Functionality Code Routings.” The report is intended to provide a full project document that can accompany source code, screenshots, test evidence, and a live demonstration. A successful submission should make it easy for an evaluator to understand the problem, see the solution, inspect the technical design, observe the functionality, and verify the results.

- Final title: PocketSmart AI: Your Smart Budget.
- Final subtitle: Main Functionality Code Routings.
- Project context: Naan Mudhalvan.
- Include project abstract.
- Include problem statement.
- Include objectives.
- Include scope.
- Include requirement analysis.
- Include system architecture.
- Include UI flow.
- Include route map.
- Include calculation formulas.
- Include database/data model if applicable.
- Include domain templates.
- Include validation rules.
- Include error handling.
- Include AI explanation design.
- Include testing strategy.
- Include test results.
- Include screenshots of working modules.
- Include source-code evidence where required.
- Include demonstration dataset.
- Include live demonstration steps.
- Include limitations.
- Include future enhancements.
- Include conclusion.
- Keep source code and report terminology consistent.
- Verify all displayed calculations before submission.
- Verify every main route before demonstration.
- Keep a backup of the project source code.
- Prepare a short presentation explaining the architecture.
- Demonstrate one complete personal budget.
- Demonstrate one additional domain.
- Show what-if functionality.
- Show validation behavior.
- Explain how AI is separated from arithmetic.
- Finish by explaining future scope.
- Final deliverable: a complete, technically explainable PocketSmart AI project package.

Code-routing principle: User input should move through validation and business services before reaching the calculation engine. The verified result should then be used consistently by the dashboard, reports, and optional AI explanation layer.

Project-quality principle: Each major function should have a defined input, processing rule, output, validation rule, and test case. This makes the implementation easier to explain and the demonstration easier to verify.